ARL Spec: ARL API

Brendan Griffiths

Contents

1 Service overview 2
1.1 General state and routines 2
2 Transmitted data 2
2.1 Researchers 2
2.2 Targets 3
2.3 Alert limits 3
2.4 Backlog 3
2.5 Health 4
3 Controls 5
4 Alerts 5
5 Configuration 6

1 Service overview

The ARL API is the laboratory's HTTP support service. It holds the researcher register (who may control which service), the RTA target catalogue, the alert limits of every service, and a backlog of everything the services have published to the broker. It does not publish to MQTT itself; the services ask it to verify researchers and read their alert limits from it.

Aspect	Details
Base path	`/v1` for the data endpoints. Interactive documentation at `/docs` (Swagger UI) and the OpenAPI document at `/openapi.json`.
Format	JSON requests and responses (the backlog export is CSV). Errors are `{"error": <code>, "detail": <string or null>}` with the matching HTTP status: `not_found` (404), `invalid_request` (400), `forbidden` (403), `too_many_rows` (413), `rate_limited` (429), `internal` (500).
Authentication	None for reads.
Live deployment	Read only: every write endpoint returns `403 forbidden`. Rate limited to 20 requests per second (burst 40) per client address, and one backlog download at a time per client; over the limit the API returns `429` with `Retry-After: 1`.
Test deployment	Targets can be created and deleted. Not rate limited.
Services	Service codes used in paths and records: `rta`, `mhe`, `lwm`, `opg`, `owt`.

1.1 General state and routines

- **Researchers**: each researcher has a code (e.g. `arl-as3`), a name, the experiments (service codes) they work on, and a status. Only `active` researchers whose experiments include a service may send that service commands; the services check this through the verify endpoint, so a command from anyone else is rejected (`unknown_researcher`, `researcher_defunct` or `researcher_not_permitted`).
- **Targets**: the catalogue of named sky targets the RTA can observe by id. Ids are `tgt-` plus three letters.
- **Alert limits**: the thresholds the services use to raise alerts, per service. The services re-read them, so the API is the single source of truth for every limit.
- **Backlog**: the API records every telemetry, state and event message published by the services (never commands or acknowledgements) and serves it as CSV, up to 7 days per request. On the live deployment the backlog also contains history from before the deployment started (for example earlier MHE mission days and LWM weather history).

2 Transmitted data

2.1 Researchers

`GET /v1/researchers/{code}` returns one researcher (`404 not_found` if the code is unknown).

Measurement	Key	Level	Type / unit	Description
Code	`code`	Researcher	string	The researcher's code, as used in command payloads.
Name	`name`	Researcher	string	The researcher's name.
Experiments	`experiments`	Researcher	array of service codes	The services the researcher works on and may control.
Status	`status`	Researcher	enum	`active` or `defunct`. A defunct researcher may not control any service.

2.2 Targets

`GET /v1/targets` lists the catalogue (optionally `?category=<category>`); `GET /v1/targets/{id}` returns one target.

Measurement	Key	Level	Type / unit	Description
Id	`id`	Target	string	Target id, e.g. `tgt-cen`. Used by the RTA Set Target control.
Name	`name`	Target	string	Common name, e.g. Centaurus A.
Right ascension	`ra_rad`	Target	float, radians, 0 to 2π	Sky position of the target's centre.
Declination	`dec_rad`	Target	float, radians, $-\pi/2$ to $\pi/2$	Sky position of the target's centre.
Field of view	`fov_rad`	Target	float, radians, 0.0005 to 0.05	Width of the region of sky an RTA scan of the target covers.
Category	`category`	Target	string	Kind of object, e.g. `radio galaxy`.

2.3 Alert limits

`GET /v1/alert-limits/{service}` returns the list of limits of a service (`404` for an unknown service). The limit values and their alerts are listed under *Alerts* below.

Measurement	Key	Level	Type / unit	Description
Key	`key`	Service	string	The limit's key within the service, e.g. `high_wind.warn_ms`. In the service sheets it is written with the service prefix (`lwm.high_wind.warn_ms`).
Value	`value`	Service	float	The limit.
Unit	`unit`	Service	string or `null`	Unit of the value, derived from the key (`K`, `m/s`, `%`, `ppm`, …). `null` for counts, ratios and scores.

2.4 Backlog

`GET /v1/backlog/{service}/streams` says what has been recorded for a service:

Measurement	Key	Level	Type / unit	Description
Telemetry streams	`telemetry`	Service	array of strings	Recorded telemetry stream names (e.g. `weather`).
State items	`state`	Service	array of strings	Recorded state item names.
Events	`events`	Service	array of strings	Recorded event kinds.
Earliest	`earliest_ts`	Service	timestamp, ms or `null`	Time of the oldest recorded message.
Latest	`latest_ts`	Service	timestamp, ms or `null`	Time of the newest recorded message.

`GET /v1/backlog/{service}.csv` downloads recorded messages as CSV, ordered by `ts` then entity. Query parameters:

Measurement	Key	Level	Type / unit	Description
Kind	`kind`	Request	enum	`telemetry`, `state` or `events`. Required.
Stream / item / event	`stream`, `item`, `event`	Request	string	The telemetry stream, state item or event kind to export (the one matching `kind` is required).
From	`from`	Request	timestamp, ms	Start of the period, inclusive. Required.
To	`to`	Request	timestamp, ms	End of the period, exclusive. Required; at most 7 days after `from`.
Entity	`entity`	Request	string	Optional: only this entity.

CSV columns (one header row): telemetry `ts,entity,` then the stream's fields in `$topology` order (empty cell for `null`); state `ts,entity,item,value`; events `ts,entity,kind,payload` (`value` and `payload` are JSON text). At most 5,000,000 rows per request (`413 too_many_rows` beyond that; narrow the period or filter by entity).

2.5 Health

`GET /healthz`:

Measurement	Key	Level	Type / unit	Description
Status	`status`	Global	enum	`ok`, or `degraded` when the database or broker connection is down.
Database	`database`	Global	boolean	Whether the database is reachable.
Broker	`broker`	Global	boolean or `null`	Whether the backlog recorder is connected to the broker (`null` when it is not recording).

3 Controls

Control	Key	Level	Value (type / unit)	Description
Verify Researcher	`POST /v1/researchers/verify`	Researcher	JSON `{code` string, `service` service code`}`	Checks whether a researcher may control a service. Returns `{valid` boolean, `reason}` where `reason` is `unknown`, `defunct` or `not_permitted` when not valid, else `null`. Read only; allowed on the live deployment.
Create Target	`POST /v1/targets`	Target	JSON `{name` 1 to 64 chars, `ra_rad` 0 to 2π, `dec_rad` −π/2 to π/2, `fov_rad` 0.0005 to 0.05, `category` 1 to 64 chars`}`	Adds a target to the catalogue and returns it (`201`) with its new id. Test deployment only (`403` on live).
Delete Target	`DELETE /v1/targets/{id}`	Target	path: target id	Removes a target (`204`, or `404` if unknown). Test deployment only (`403` on live).

4 Alerts

The API raises no alerts itself. It serves the limits every service uses for its alerts (`GET /v1/alert-limits/{service}`). Defaults:

Service	Key	Default	Unit	Used by alert
`rta`	`availability.min_active`	80	dishes	`availability`: minimum dishes in service
`rta`	`wind_stow.gust_ms`	15	m/s	`wind_stow`: gust that engages wind stow
`rta`	`wind_stow.release_ms`	10	m/s	`wind_stow`: gust below which it releases
`rta`	`dish_temperature.warn_k`	333	K	`dish_temperature` warning
`rta`	`dish_temperature.crit_k`	343	K	`dish_temperature` critical
`rta`	`dish_power.max_w`	5000	W	`dish_power`
`rta`	`calibration.max_age_h`	168	h	`dish_calibration` (overdue)
`rta`	`scan.min_snr_db`	3	dB	`dish_scan` (low SNR)
`mhe`	`crew.hr_min`, `crew.hr_max`	40, 150	bpm	`crew_health`: heart rate
`mhe`	`crew.temp_max_k`	311.15	K	`crew_health`: body temperature
`mhe`	`crew.resp_max`	30	breaths/min	`crew_health`: respiratory rate
`mhe`	`crew.spo2_min`	92	%	`crew_health`: blood oxygen
`mhe`	`survey.min_mood`	4	score	`survey_wellbeing`: 3-day mean mood
`mhe`	`survey.max_stress`	7	score	`survey_wellbeing`: 3-day mean stress
`mhe`	`survey.max_decline`	2	score	`survey_wellbeing`: mood drop over 7 days
`mhe`	`room.temp_min_k`, `room.temp_max_k`	290, 298	K	`room_condition`: temperature

Service	Key	Default	Unit	Used by alert
mhe	room.humidity_min, room.humidity_max	25, 75	%	room_condition: humidity
mhe	room.humidity_max_hydroponics	85	%	room_condition: humidity in hydroponics
mhe	air.o2_min_ppm	195000	ppm	air_concentration: oxygen
mhe	air.co2_warn_ppm, air.co2_crit_ppm	1500, 2500	ppm	air_concentration: CO_2 warning, critical
lwm	high_wind.warn_ms, high_wind.crit_ms	15, 20	m/s	high_wind warning, critical
lwm	lightning.max_km	10	km	lightning_proximity
lwm	storm.approach_km	25	km	storm_approach
lwm	temperature.min_k, temperature.max_k	268, 313	K	extreme_temperature
lwm	rain.max_mm_h	10	mm/h	heavy_rain
lwm	battery.min_v	11.5	V	low_battery
lwm	calibration.max_days	30	days	calibration_overdue
lwm	soiling.min_ratio	0.95	ratio	soiling (site lwm-4)
lwm	pan.min_mm, pan.max_mm	175, 230	mm	pan_level (site lwm-5)
opg	solar.min_ratio	0.8	ratio	solar_underperformance
opg	battery.warn_pct	30	%	low_battery warning
opg	battery.max_temp_k	318	K	battery_temperature
opg	fuel.min_pct	25	%	low_fuel
owt	intake.max_pct	95	%	intake_full
owt	quality.max_ntu	1.0	NTU	water_quality: turbidity
owt	quality.ph_min, quality.ph_max	6.5, 8.5	pH	water_quality: pH
owt	filter.min_pct	20	%	filter_clogged
owt	recycling.min_pct	70	%	low_recycling_efficiency

The unit column is the quantity's unit; the API's `unit` field is `null` for keys without a unit suffix (counts, ratios, scores, pH). Alerts with no limit (offline, fault and missed-survey alerts, and the OWT reserve, which is the `storage_reserve` control) do not appear here.

5 Configuration

The API reads a TOML file (`config/api.toml` in the bundle, `--config` or `ARL_CONFIG` for the binary). Environment variables override the file, and command-line options override both. Defaults are the values in the provided file.

Key	Type / unit	Default	Description
mode	enum	"test"	`test` for your local copy (targets can be created and deleted, no rate limit) or `live` (read only, rate limited). Env `API_MODE`, option `--mode`.
port	integer	8080	HTTP port. Env `ARL_API_PORT`, option `--port`.

Key	Type / unit	Default	Description
`database_url`	string	`"postgres://arl:arl@postgres/arl"`	PostgreSQL connection URL. Env `DATABASE_URL`, option `--database-url`.
`seed_dir`	string, path	`"/seed"`	Folder of the initial researchers, targets and alert limits, loaded into an empty database. Env `ARL_SEED_DIR`, option `--seed-dir`.
`retention_days`	integer, days	`75`	How long recorded backlog data is kept.
`record`	boolean	`true`	Whether the API records the broker into the backlog.
`mqtt.address`	string	`"mosquitto"`	MQTT broker host for the recorder. Env `MQTT_BROKER_ADDRESS`.
`mqtt.port`	integer	`1883`	MQTT broker port. Env `MQTT_BROKER_PORT`.
`mqtt.user`	string	`""`	Broker username, empty for anonymous. Env `MQTT_BROKER_USER`.
`mqtt.password`	string	`""`	Broker password. Env `MQTT_BROKER_PASSWORD`.
`health.port`	integer	`9100`	Port of the internal health endpoint used by `healthcheck`. Env `HEALTH_PORT`.

The binary also needs a reachable PostgreSQL database (`DATABASE_URL`); the provided compose file runs one for you. `docker compose -f arl-test.yaml down -v` deletes the database, so the seed data is loaded again on the next start.

AI Declaration: Claude Code (Opus 5.5) - generated from the arl-sim source, reviewed